CREO: a repair-displacement term for population-based search, and the boundary of its usefulness

Alan Immanuel Benjamin Vallavaraj
a.vallavaraj@westminster.ac.uk

October 20, 2026

Abstract

A recurring idea in constrained and bounded optimisation is that the correction a repair operator applies to an infeasible candidate is informative: it points away from the violated region, and so reusing it might guide the search. We test that idea as an algorithm component rather than as an algorithm. The component, which we call CREO, for Continuous Repair Evolutionary Optimiser, is a single term $\beta\,\Delta r$ added to the update rule of eight population-based optimisers, where Δr is the displacement the repair operator applied to that individual at its last evaluation. At $\beta = 0$ the arm is stream-identical to the unmodified host, so the comparison is a within-host ablation in which a matched pair differs by one scalar and nothing else. We run 29,760 matched runs over twelve nonlinearly constrained problems with five repair operators spanning distinct displacement geometries, and 46,080 matched runs of a shifted-optimum control in which each landscape appears twice, differing only in whether its optimum sits at the domain centre. Three results follow. First, the term is net harmful: for every one of the five operators it is significantly worse in more host-by-problem cells than it is better, and all eight hosts are net harmed. Second, the apparent benefit of the radial-shrinkage operator, the geometry used by a widely adopted benchmark protocol, is a centre-seeking artefact: it helps when the optimum lies at the domain centre and hurts when the same landscape is translated away from it, with a median centre advantage of $+0.368$ in $\log_{10}$ gap ratio at $D = 30$. Third, the natural explanation that a displacement helps in proportion to the geometric information it carries is falsified: the ellipsoid-projection operator, whose displacement is a true surface normal with D degrees of freedom, is the most harmful of the five. Finally, a third campaign of 19,200 runs tests the remaining conjecture, that the mechanism belongs to tracking rather than to static search, by letting the optimum drift at four rates with the static case retained as an internal control; the conjecture is refuted in the direction opposite to the one predicted, the component becoming more harmful as the target moves faster. Against this, we report one environment in which the term demonstrably does work: a deployed rate-constrained set-point tracking loop, where a sweep of β against a stream-identical zero-term baseline gives a strictly monotone dose-response and the term accounts for 78% of the configuration's advantage over its bare host. We could not identify the structural property responsible: reproducing the loop's repair geometry, limit magnitude, budget, dimensionality, objective rank-deficiency, plant lag and long-traverse initialisation, singly and in combination, never made the term beneficial. We conclude that displacement reuse is not a general search signal, that benchmark results for this family should be read as confounded unless a shifted-optimum control is reported alongside them, and that the dissociation between static and closed-loop behaviour is a concrete open problem rather than a resolved one.

Keywords. algorithm components; repair operators; constraint handling; within-host ablation; benchmark bias; centre-bias; differential evolution; grey wolf optimiser; CMA-ES; rate-limited control; negative results.

1 Introduction

The optimisation literature has accumulated a large number of algorithms whose novelty is metaphorical rather than structural, and a sustained critique has argued that the unit of analysis should be the *component* rather than the named algorithm [Sörensen, 2015, Camacho-Villalón et al., 2022]. A component contribution is specified as a term in an update rule, evaluated by ablation inside hosts that are otherwise unchanged, and reported with the conditions under which it helps and the conditions under which it does not.

This paper applies that standard to one specific idea. When a candidate solution violates a constraint or leaves the search domain, a repair operator moves it to an admissible point. The vector between the proposed point and the repaired point, which we write Δr, is ordinarily discarded. It is tempting to suppose that it is informative: geometrically it points out of the violated region, so a search that remembers it might avoid re-proposing the same violation, or might infer something about where the admissible region lies. Mechanisms of this kind recur in the constrained-optimisation literature under various names.

We test the idea directly, and the answer is negative in a way that is more informative than a simple absence of effect. The mechanism is not merely neutral; it is harmful, and the one setting in which a benefit has been reported for an operator of this family turns out to be explained by a property of the benchmark rather than by the mechanism.

Contributions.

1. A component specification of displacement reuse with an exact zero-term baseline: at $\beta = 0$ the arm consumes no random numbers and is stream-identical to the unmodified host, verified for all eight hosts by an automated test.

2. A multi-geometry ablation over eight hosts, five repair operators and twelve constrained problems (29,760 matched runs), showing the term is net harmful under every geometry tested, including a null operator whose displacement is uninformative by construction.

3. A shifted-optimum control (46,080 matched runs) that measures, rather than argues, the centre-pull confound: the same landscape is optimised twice, differing only in whether the optimum sits at the domain centre.

4. A falsification of the obvious mechanistic explanation. If displacement reuse worked because the displacement carries geometric information, the operator carrying the most information would help the most. It harms the most.

5. A moving-target control (19,200 matched runs) that tests, and refutes, the remaining conjecture that the mechanism belongs to tracking rather than to static search. The static case is retained as an amplitude-zero arm inside the same grid, so the comparison is internal.

2 The component

Let a host algorithm propose, for individual i, a candidate by its own update rule $\mathcal{U}_{\text{host}}$. The component adds exactly one term:

$$x_i \;=\; \mathcal{U}_{\text{host}}(\cdot) \;+\; \beta\,\Delta r_i, \qquad \Delta r_i \;=\; x_i^{\text{used}} - x_i^{\text{stored}}, \tag{1}$$

where x_i^{stored} is the point held in the population and x_i^{used} is the point actually evaluated after repair. For DE/rand/1/bin the complete rule reads

$$x_i \;=\; x_{r_1} + F\,(x_{r_2} - x_{r_3}) \;+\; \beta\,\Delta r_i, \tag{2}$$

which differs from the canonical rule [Storn and Price, 1997] by the final term alone, so a reader can diff the two in one line.

We name this term CREO, for *Continuous Repair Evolutionary Optimiser*, so that it can be cited and built on. The name denotes the term in Equation 1 and nothing else: it is not an algorithm, and a host carrying it is a *configuration* of that host rather than a new method. Earlier work by the author used the expansion "Repair-Guided Evolutionary Optimisation" for the same term; the present expansion is adopted here because it matches the acronym, and the two refer to the identical mechanism. We write "DE with CREO" or "$\beta > 0$" in preference to constructions such as "CREO-DE", because a roster of hyphenated names implies a family of algorithms where there is one coefficient.

Two properties make this an ablation rather than a comparison of algorithms. First, the term is additive and draws no random numbers, so setting $\beta = 0$ recovers the unmodified host *exactly*, including its random stream; this is asserted for all eight hosts by a test that forces $\Delta r \equiv 0$ and requires bit-identical trajectories. Second, the repair operator is a property of the environment and is applied identically in both arms, so a matched pair of runs differs by the value of β and by nothing else. We hold $\beta = 0.90$ throughout and do not tune it; unbalanced tuning effort between a proposed component and its baseline is a known failure mode, and leaving the coefficient fixed is the conservative choice in a study whose finding is negative.

Algorithm 1 states the component in full, inside a generic host loop. The only line that differs from the unmodified host is line 8; every other line, including all random draws, is the host's own. Setting $\beta = 0$ therefore recovers the host exactly rather than approximately, which is what licenses the matched pairing used throughout.

Algorithm 1 Displacement reuse as a host-agnostic component. $\mathcal{U}_{\text{host}}$ is the host's unmodified proposal rule; $\mathcal{R}$ is the repair operator; $\prec$ is the host's own feasibility-aware comparison. The quantity Δr is the only state the component adds, one vector per individual.

Require: host $\mathcal{U}_{\text{host}}$, repair operator $\mathcal{R}$, coefficient β, budget B, problem $(f, g, \text{lo}, \text{hi})$

1: initialise population $P = \{x_1, \ldots, x_N\}$ uniformly in $[\text{lo}, \text{hi}]$
2: **for** $i = 1 \ldots N$ **do**
3: $\quad \Delta r_i \leftarrow \mathbf{0}$ ▷ no predecessor exists at initialisation, so no repair is applied
4: **end for**
5: **while** evaluations used $< B$ **do**
6: $\quad$ **for** each individual i **do**
7: $\quad\quad \tilde{x} \leftarrow \mathcal{U}_{\text{host}}(P, i)$ ▷ host proposal, consuming the host's random draws
8: $\quad\quad \tilde{x} \leftarrow \tilde{x} + \beta\, \Delta r_i$ ▷ **the component: one added term, no random draws**
9: $\quad\quad x^{\text{stored}} \leftarrow \text{clip}(\tilde{x}, \text{lo}, \text{hi})$
10: $\quad\quad x^{\text{used}} \leftarrow \text{clip}(\mathcal{R}(x^{\text{stored}}, x_i^{\text{prev}}), \text{lo}, \text{hi})$ ▷ repair; identical in both arms
11: $\quad\quad \Delta r_i \leftarrow x^{\text{used}} - x^{\text{stored}}$ ▷ retain the displacement for the next iteration
12: $\quad\quad$ evaluate $f(x^{\text{used}})$ and violation $v(x^{\text{used}})$
13: $\quad\quad c \leftarrow (x^{\text{stored}}, x^{\text{used}}, \Delta r_i, f, v)$
14: $\quad\quad$ **if** $c \prec P_i$ **then**
15: $\quad\quad\quad P_i \leftarrow c$ ▷ host selection, unchanged
16: $\quad\quad$ **end if**
17: $\quad$ **end for**
18: **end while**
19: **return** best individual by $\prec$

Three details in Algorithm 1 are load-bearing and are easy to get wrong. First, the displacement is measured against the *stored* point, not the proposal, so that domain clipping is attributed to the repair operator rather than to the component. Second, Δr_i is read at the start of an iteration and written at the end, so the term always uses the previous iteration's

displacement; using the current one would make the update implicit. Third, the operator receives x_i^{prev}, the incumbent the candidate was generated from. Omitting that argument makes a rate-limiting operator a silent no-op, because a rate limit is undefined without a predecessor: the arm then returns ties for every run and reads as a null result rather than as a bug. We check for this explicitly before each campaign.

3 Repair operators

The central question is whether the usefulness of Δr depends on the *geometry* of the operator that produced it. Five operators are therefore used, spanning distinct displacement geometries, each parameterised on the search box:

Radial shrinkage. Contracts the candidate towards the domain centre outside a soft region. Its displacement has one degree of freedom, its magnitude, and always points at the centre. This is the geometry used by the preferred-region protocol of a widely adopted benchmark suite.

Box projection. Componentwise projection onto a soft box. The displacement is axis-aligned and encodes only which coordinates were violated.

Ellipsoid projection. Projection onto a randomly oriented ellipsoid, fixed per problem so the geometry is part of the problem rather than part of the run. The displacement is a true surface normal with D degrees of freedom, and is therefore the most geometrically informative operator in the battery.

Rate limit. Bounds each coordinate's change from the individual's predecessor, so the displacement is exactly the component of the requested step that the limiter refused. Of the five, this is the operator whose displacement most directly encodes a direction the search *wanted* to move.

Random resample. A null control: a violating candidate is replaced by a fresh uniform sample, so the displacement is uninformative by construction. Any benefit here is attributable to perturbation magnitude alone, not to guidance.

The rate limit requires a predecessor. We use the incumbent each candidate is generated from: the target vector for the differential-evolution variants and for the grey-wolf and ε-constrained hosts, the child's own parent for the genetic algorithm, and the particle's current position for particle swarm. CMA-ES has no per-individual predecessor, so we use the distribution mean; we flag this as an analogy rather than a true predecessor. The initial population is left unrepaired, since sampling has no predecessor.

4 Hosts and problems

Eight hosts span algorithmic-component space rather than metaphor space: DE/rand/1/bin and DE/current-to-$p$best/1 [Storn and Price, 1997], L-SHADE [Tanabe and Fukunaga, 2014], SPSO-2011 [Zambrano-Bigiarini et al., 2013], a real-coded genetic algorithm with simulated binary crossover and polynomial mutation, the Grey Wolf Optimiser [Mirjalili et al., 2014], ε-constrained differential evolution [Takahama and Sakai, 2009], and CMA-ES [Hansen and Ostermeier, 2001]. The Grey Wolf Optimiser is included because it is widely used in the applied literature, with the explicit acknowledgement that its update rule has been shown to be a reiteration of particle-swarm ideas rather than a novel mechanism [Camacho-Villalón et al., 2022]; it appears here as a host, not as a contribution. Selection is feasibility-aware in every host, following Deb's rules [Deb, 2000], so that feasibility handling is identical across arms.

The constrained campaign uses twelve nonlinearly constrained problems from the standard G-suite together with two engineering design problems, as implemented in `pymoo` [Blank and Deb, 2020]. The shifted-optimum control uses the 24 BBOB functions [Hansen et al., 2021] through the `IOHexperimenter` interface [de Nobel et al., 2023]. All runs use a budget of 2×10^4 function evaluations.

5 Experimental design

Constrained campaign. Eight hosts × five operators × twelve problems × 31 paired seeds × two arms gives 29,760 runs. The repair operator acts on the search domain; the problems' own nonlinear constraints are handled by feasibility-aware selection, identically in both arms.

Shifted-optimum control. This is the decisive test of the centre-pull confound. Each BBOB function, at instances 1–5 and $D \in \{10, 30\}$, is run in two variants that differ *only* in where the optimum sits: *translated*, $f(x)$, with the optimum at $x^\star$ away from the domain centre, and *centred*, $f(x + x^\star)$, the same landscape with the optimum at the origin, which is the centre of the box. Three operators × two seeds × two arms gives 46,080 runs. An operator whose displacement points at the domain centre must help in the centred variant and not in the translated one. If it does, a benefit reported on a suite whose optima are biased towards the centre is a centre-seeking artefact rather than repair guidance [Kůdela, 2022].

Moving-target control. The static campaigns leave one conjecture open: that displacement reuse belongs to *tracking* rather than to repair geometry. We test it inside a single design. Each of six BBOB functions, spanning separable to highly multimodal, is optimised at $D \in \{10, 30\}$ with its optimum displaced along a deterministic trajectory

$$s(t) \;=\; A\,\tfrac{1}{2}(\mathrm{hi} - \mathrm{lo})\,\sin(2\pi t/T)\,\hat{d}, \qquad T = B/4, \tag{3}$$

where t is the evaluation index, $\hat{d}$ a unit direction fixed per function, and A the drift amplitude as a fraction of the domain half-width. Crucially $A = 0$ reproduces the static case, so the control is *internal* to the campaign rather than a comparison across campaigns. Eight hosts × five operators × six functions × two dimensions × four amplitudes × five seeds × two arms gives 19,200 runs. Because the schedule depends on t alone it is identical in both arms of a matched pair. Best-so-far objective value is meaningless when the optimum moves, so the reported measure is *tracking error*: the mean distance from each evaluated point to the optimum's current location, normalised by the domain width.

Statistics. Arms are paired exactly on host, operator, problem, dimension and seed. On the constrained suite, runs are ordered by the feasibility-aware key, matching the selection rule used inside the hosts; the objective is compared by Wilcoxon signed-rank over the 31 seeds within each host-by-operator-by-problem cell, restricted to runs feasible in both arms, and cells with fewer than six such pairs are reported as untested rather than silently dropped. On the shifted-optimum control the paired quantity is $\log_{10}$ of the ratio of optimality gaps, positive favouring the component, and the confound is tested by comparing the centred and translated improvements for the same landscape, which are themselves a matched pair. All p-values are Holm-corrected within their family [Holm, 1979].

6 Results

6.1 The term is net harmful under every geometry

Table 1 aggregates the 480 host-by-problem cells, 96 per operator. For every operator the term is significantly worse in more cells than it is better. The best case is box projection at six better against seventeen worse. The rate-limit operator, whose displacement most directly encodes a refused direction, is never significantly better in any of its 96 cells.

Table 1: Constrained suite. Cells are host × problem, 96 per operator; each cell is a Wilcoxon signed-rank test over 31 paired seeds, Holm-corrected. "Untested" marks cells with fewer than six seed pairs feasible in both arms. Matched-pair counts are over all 14,880 pairs.

Repair operator	Cells of 96			Untest.	Matched pairs		
	better	worse	n.s.		wins	ties	losses
box projection	6	17	44	29	900	454	1622
ellipsoid projection	5	31	13	47	519	30	2427
radial shrinkage (CEC2017)	3	38	31	24	560	78	2338
random resample (null)	4	14	40	38	656	1205	1115
rate limit (E06)	0	14	61	21	970	218	1788

The effect is not confined to particular hosts. Table 2 shows that all eight are net harmed, the genetic algorithm coming closest to neutral at three cells better against four worse.

Table 2: Constrained suite, aggregated by host over all five operators and twelve problems.

Host	Cells better	Cells worse	Pair wins	Pair losses
CMA-ES	0	14	478	1181
DE/current-to-pbest/1	1	22	224	1444
DE/rand/1/bin	0	21	182	1300
GA (SBX + polynomial)	3	4	839	1006
GWO	3	18	371	1294
L-SHADE	3	15	496	1125
SPSO-2011	5	9	611	880
eps-constrained DE	3	11	404	1060

Two observations bear on the mechanism. The null operator, whose displacement is a fresh uniform sample and therefore carries no guidance at all, is not the worst of the five: it is better than radial shrinkage and ellipsoid projection on significant-cell counts and has by far the most ties. And the ellipsoid operator, the most geometrically informative, fires on 99.7% of candidates and is the most harmful. Firing frequency does not explain the ordering either: the rate-limit operator fires on 72.1% of candidates and is the mildest.

6.2 The centre-pull confound, measured

Table 3 reports the shifted-optimum control. For radial shrinkage the pattern is unambiguous and large: the component helps when the optimum sits at the domain centre and hurts when the identical landscape is translated away from it. The median centre advantage is +0.178 in $\log_{10}$ gap ratio at $D = 10$ and +0.369 at $D = 30$, with the centred variant better in 66.4% and 79.7% of matched landscapes respectively.

Note that the centre advantage is the median of the paired differences, not the difference of the two medians, and the two need not agree: at $D = 30$ the medians differ by 0.179 while the median paired difference is 0.369. The paired quantity is the correct one here, because each

Table 3: Shifted-optimum control. Values are medians of the paired $\log_{10}$ gap ratio (positive favours the component); "centre adv." is the median of (centred − translated) over matched landscapes and "frac." the fraction of landscapes in which the centred improvement is larger. $n = 1920$ matched landscapes per row.

Repair operator	D	centred	transl.	centre adv.	frac.	Holm p
box projection	10	+0.0000	+0.0000	+0.0000	0.472	0.7166
box projection	30	+0.0000	-0.0005	+0.0186	0.553	0.0003
ellipsoid projection	10	-0.2135	-0.0076	-0.1432	0.279	8.12×10^{-96}
ellipsoid projection	30	-0.1276	-0.0261	-0.0608	0.248	6.11×10^{-105}
radial shrinkage (CEC2017)	10	+0.0033	-0.0748	+0.1785	0.664	7.34×10^{-58}
radial shrinkage (CEC2017)	30	+0.0703	-0.1091	+0.3687	0.797	2.51×10^{-175}

landscape contributes a centred and a translated run of the same function at the same instance, dimension and seed.

Box projection shows no effect in either variant. Ellipsoid projection is harmful in both, and more harmful in the centred variant, which is the opposite of a centre-pull signature and consistent with its displacement actively fighting the host's own step. Table 4 gives the per-variant effects in full.

Table 4: Shifted-optimum control, effect of the component by operator, optimum location and dimension. "fire rate" is the mean fraction of candidates the repair operator modified.

Repair operator	Variant	D	median ratio	better	fire rate	Holm p
box projection	centred	10	+0.0000	0.432	0.352	0.7293
box projection	centred	30	+0.0000	0.491	0.595	0.0912
box projection	translated	10	+0.0000	0.443	0.638	0.4984
box projection	translated	30	-0.0005	0.469	0.878	0.2805
ellipsoid projection	centred	10	-0.2135	0.202	1.000	5.92×10^{-113}
ellipsoid projection	centred	30	-0.1276	0.178	1.000	2.64×10^{-144}
ellipsoid projection	translated	10	-0.0076	0.194	1.000	1.39×10^{-122}
ellipsoid projection	translated	30	-0.0261	0.194	1.000	5.18×10^{-130}
radial shrinkage (CEC2017)	centred	10	+0.0033	0.510	0.504	1.19×10^{-12}
radial shrinkage (CEC2017)	centred	30	+0.0703	0.613	0.800	3.64×10^{-26}
radial shrinkage (CEC2017)	translated	10	-0.0748	0.345	0.862	9.16×10^{-32}
radial shrinkage (CEC2017)	translated	30	-0.1091	0.321	0.961	1.82×10^{-52}

6.3 The tracking conjecture, tested and refuted

If the mechanism's premise is that a refused step describes a constraint the search is actively fighting, then its value should rise as the target moves. It does not. Table 5 reports the effect at each drift amplitude: at every level between five and six of the ten operator-by-dimension cells are significantly harmful and *none* is significantly helpful, and the median benefit becomes more negative as the target moves faster, from -0.0028 static to -0.0046 at $A = 0.20$.

Table 6 tests the conjecture directly, comparing each configuration's benefit when the target moves against the same configuration's benefit when it is static. The drift advantage is zero to five decimal places at every amplitude, the moving case is better in 48.5% to 49.8% of matched configurations, which is a coin flip, and Holm-corrected $p = 1.0$ throughout. Restricting to the rate-limit operator, the geometry whose displacement is precisely a refused step, does not rescue it: at the largest amplitude the component is *significantly worse* than in the static case (-0.0017, Holm $p = 0.039$).

Table 5: Moving-target control. Cells are operator × dimension, ten per drift amplitude; "benefit" is the reduction in normalised tracking error attributable to the component, so positive favours it. $A = 0$ is the static control internal to this campaign.

Drift A	Cells	Sig. harmful	Sig. helpful	Median benefit	Helped frac.
0.00	10	5	0	-0.00280	0.404
0.01	10	6	0	-0.00390	0.402
0.05	10	5	0	-0.00347	0.429
0.20	10	6	0	-0.00463	0.398

Table 6: The tracking conjecture. "Drift adv." is the median of (benefit when moving − benefit when static) over configurations matched on host, operator, function, dimension and seed. A positive value would support the conjecture.

Scope	A	n	Static benefit	Moving benefit	Drift adv.	Holm p
all operators	0.01	2400	-0.00096	-0.00133	+0.00000	1.0000
all operators	0.05	2400	-0.00096	-0.00122	+0.00000	1.0000
all operators	0.20	2400	-0.00096	-0.00167	+0.00000	1.0000
rate limit only	0.01	480	—	-0.00701	+0.00000	0.5544
rate limit only	0.05	480	—	-0.00660	-0.00000	0.9717
rate limit only	0.20	480	—	-0.00980	-0.00167	0.0393

The conjecture is therefore refuted on its own terms, and refuted in the direction opposite to the one predicted: target motion makes the component worse, not better.

7 A deployed closed loop, where the term does work

Every result so far is negative. We now report the one environment in which the term demonstrably helps, because the contrast is the most informative part of this study and because a mechanism paper that reported only the negative half would be as selective as one reporting only the positive half.

The environment is a rate-constrained set-point tracking loop: a controller proposes a six-dimensional bounded control each cycle, a rate limiter clips it to ±0.10 of range about the previously applied control, a two-dimensional plant with first-order lag responds, and the score is the distance of the plant state from a target integrated over fifteen cycles, at twenty evaluations per cycle, giving a 300-evaluation budget. The repair operator is therefore exactly the rate-limit geometry of Section 3, and the displacement is exactly the refused component of the requested control change. Each arm is run over 67,200 matched episodes across 21 stress profiles.

7.1 An isolated dose-response in β

The decisive measurement is a sweep of β within one controller, holding every other term fixed. In this implementation the component block executes whenever repair guidance is enabled, irrespective of the value of β, so the $\beta = 0$ arm consumes the identical random stream: it is an exact zero-term baseline of the kind Section 3 requires. Table 7 reports the result.

Distance AUC decreases strictly monotonically across all seven levels and success rises with it, from 0.578 at $\beta = 0$ to 0.659 at $\beta = 1.50$, and the effect has not saturated at the largest value tested. Decomposing the gain of the deployed configuration over the bare host on this condition: the total improvement in AUC is 3.72%, of which 0.84% is attributable to the other differences between the two arms and 2.91% to β alone, so the displacement term accounts for 78.1% of the

Table 7: Dose-response in β on the closed loop, baseline stress condition, all other coefficients held fixed. The $\beta = 0$ row is the stream-identical zero-term baseline.

β	Distance AUC ↓	Success ↑	Repair rate
0.00	0.421479	0.578438	0.876975
0.25	0.421147	0.573125	0.873916
0.50	0.417020	0.590313	0.879507
0.75	0.412328	0.592187	0.897020
0.90	0.409232	0.608437	0.921280
1.10	0.403673	0.638437	0.975349
1.50	0.397247	0.659062	0.999878

effect. On this environment the mechanism works, it works through the term under study, and the evidence is a dose-response rather than a single comparison.

7.2 What the ranking does and does not say

Table 8 gives the mean rank of twelve arms over the 21 profiles. The term improves every host it is attached to, in every profile. It does not, however, reorder the field: four of the six component-equipped arms still rank below the strongest bare host, and the relative gain is strongly inversely related to host quality (Spearman +0.83 between a host's own AUC and the gain the term delivers on it). The correct reading is that the term improves its host, that it improves a weak host more than a strong one, and that host selection still dominates component selection.

Table 8: Mean rank over 21 stress profiles, twelve arms. Lower is better.

Arm	Mean rank	Arm	Mean rank
creo_gwo	1.0000	pso	6.7619
creo_dfo	2.0476	creo_ga	7.5714
gwo	2.9524	creo_jso	9.0000
creo_pso	4.0000	jso	10.0000
creo_de	5.1905	ga	11.0000
dfo	6.4762	de	12.0000

7.3 An unexplained dissociation

The term helps here and harms everywhere else in this paper, and we were unable to find the reason. We reproduced, one at a time and in combination, every structural feature of this loop we could identify: the rate-limit geometry and its exact limit magnitude, the 300-evaluation budget, the six-dimensional bounded control space, a rank-deficient objective matching the loop's six-to-two map, the first-order plant lag, a moving target at four rates, and a plant initialised on the opposite side of its target so that each episode begins with a long rate-limited traverse. In none of these did the term become beneficial; the rank-deficient, rate-limited, lagged condition that most closely matches the loop is in fact the most harmful cell in the entire study (median benefit -0.0157, helped in 23.9% of matched pairs).

One structural difference remains, which we name but did not test. In the loop the rate limit is applied about a *single* applied control that advances once per cycle, so the whole population is confined to one moving box and every individual's displacement is the refused part of a step from a *shared* operating point. In our replications each individual is limited about its own predecessor, so the accumulated displacements point in as many directions as there are individuals. That

difference turns Δr from a per-individual quantity into a population-coherent one, and it is the candidate we would test next.

Until that is settled, the honest summary is narrow: the term is a usable signal in at least one deployed rate-constrained tracking loop, it is harmful on static benchmark problems under five repair geometries, and the structural property that separates the two has not been identified.

8 Discussion

What is falsified. The intuition motivating displacement reuse is that Δr carries directional information about the admissible region. If that were the operative mechanism, benefit should increase with the information content of the displacement. It does not. Ordering the operators by degrees of freedom in their displacement gives radial shrinkage (one, a magnitude), box projection (axis-aligned, sparse), and ellipsoid projection (D, a true surface normal); the ranking by harm runs in the opposite direction to the ranking by information. A null operator whose displacement is pure noise outperforms two of the three informative ones. Whatever the term is doing, it is not conveying constraint geometry to the search.

What the benchmark result was. Where a benefit for a radial-shrinkage-style operator has been reported on a standard suite, the shifted-optimum control explains it without reference to repair at all. The displacement points at the domain centre; on a suite whose optima are disproportionately near the centre, a centre pull is a free gain, and the same pull is a penalty once the optimum is moved. This stacks with the known centre-bias of several widely used benchmark suites [Kůdela, 2022]: a mechanism that happens to bias search towards the origin will appear to work, and will stop working on any instance set that does not share that bias. We would suggest that any mechanism whose displacement has a systematic direction in the search box should report a shifted-optimum control as a matter of course.

What remains open. The three defences of the mechanism *as a constraint-geometry device* have been tested and have failed: that the displacement carries useful geometry (falsified by the ellipsoid operator being the most harmful), that the reported benchmark benefit reflects repair guidance (explained instead by domain-centre bias), and that it belongs to tracking rather than static search (refuted, with harm increasing as the target moves faster).

What survives is harder and more interesting. Section 7 establishes, with a monotone dose-response against an exact zero-term baseline, that the term does real work in one deployed closed loop. Eight attempts to reproduce the conditions of that loop on standard problems all failed to make it beneficial. So we are left with a mechanism that is reliably harmful in every controlled setting we could construct and reliably helpful in the uncontrolled one it was designed for, and we cannot yet say why. We name the population-coherence of the displacement as the leading candidate: a rate limit about a single shared operating point makes every individual's Δr a component of the same refused step, whereas a per-individual limit makes them point in unrelated directions.

We therefore do not claim that no instance of this family can help. We claim that it does not help for the reasons usually given, that two of the benefits previously reported for it are confounded, and that the one benefit we can confirm is not reproduced by any controlled variation we attempted.

9 Threats to validity

The coefficient β is fixed at 0.90 and untuned; a tuned coefficient might reduce the harm, though it is hard to see how tuning recovers a benefit from a term whose informative variant is the

most harmful. A single evaluation budget is used, and the dimensional trend visible in the shifted-optimum control suggests budget and dimension interact. The predecessor mapping required by the rate-limit operator is a design choice, and the CMA-ES mapping in particular is an analogy; a different mapping might change that operator's behaviour, although it is already the mildest of the five. A substantial number of constrained cells are untested because too few seed pairs were feasible in both arms, between 21 and 47 of 96 depending on operator, so the constrained result speaks to the problems where feasibility was attainable within budget rather than to the suite as a whole. Finally, seven of the eight hosts reproduce their repair fire rates bit-identically across the two platforms we ran on, but CMA-ES does not, because the reference implementation differs in version; that arm should be reproduced only against a pinned version.

10 Reproducibility

Code, the submission script, the per-cell results for all 480 cells and the aggregate tables reported here are available at https://github.com/alanvallavaraj/CREO-X. The campaigns are gated: an equivalence test asserting that $\beta = 0$ reproduces the unmodified host, a liveness check asserting that every operator actually fires on every host, and for the moving-target campaign a check that the drift schedule measurably changes the tracking problem, all run before any compute. Both campaigns assert full row count and arm coverage before writing output. The aggregate tables reported here recompute exactly from the committed per-cell table.

References

Julian Blank and Kalyanmoy Deb. pymoo: Multi-objective optimization in python. *IEEE Access*, 8:89497–89509, 2020. doi: 10.1109/ACCESS.2020.2990567.

Christian Leonardo Camacho-Villalón, Marco Dorigo, and Thomas Stützle. Exposing the grey wolf, moth-flame, whale, firefly, bat, and antlion algorithms: six misleading optimization techniques inspired by bestial metaphors. *International Transactions in Operational Research*, 30(6):2945–2971, 2022. doi: 10.1111/itor.13176.

Jacob de Nobel, Furong Ye, Diederick Vermetten, Hao Wang, Carola Doerr, and Thomas Bäck. IOHexperimenter: Benchmarking platform for iterative optimization heuristics. *Evolutionary Computation*, 32(3):205–210, 2023. doi: 10.1162/evco_a_00342.

Kalyanmoy Deb. An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering*, 186(2–4):311–338, 2000. doi: 10.1016/S0045-7825(99)00389-8.

Nikolaus Hansen and Andreas Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195, 2001. doi: 10.1162/106365601750190398.

Nikolaus Hansen, Anne Auger, Raymond Ros, Olaf Mersmann, Tea Tušar, and Dimo Brockhoff. COCO: a platform for comparing continuous optimizers in a black-box setting. *Optimization Methods and Software*, 36(1):114–144, 2021. doi: 10.1080/10556788.2020.1808977.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Jakub Kůdela. A critical problem in benchmarking and analysis of evolutionary computation methods. *Nature Machine Intelligence*, 4:1238–1245, 2022. doi: 10.1038/s42256-022-00579-0.

Seyedali Mirjalili, Seyed Mohammad Mirjalili, and Andrew Lewis. Grey wolf optimizer. *Advances in Engineering Software*, 69:46–61, 2014. doi: 10.1016/j.advengsoft.2013.12.007.

Kenneth Sörensen. Metaheuristics—the metaphor exposed. *International Transactions in Operational Research*, 22(1):3–18, 2015. doi: 10.1111/itor.12001.

Rainer Storn and Kenneth Price. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359, 1997. doi: 10.1023/A:1008202821328.

Tetsuyuki Takahama and Setsuko Sakai. Efficient constrained optimization by the ε constrained differential evolution using an approximation model. *Transactions of the Japanese Society for Artificial Intelligence*, 24(1):34–45, 2009. doi: 10.1527/tjsai.24.34.

Ryoji Tanabe and Alex S. Fukunaga. Improving the search performance of SHADE using linear population size reduction. In *IEEE Congress on Evolutionary Computation (CEC)*, 2014. doi: 10.1109/CEC.2014.6900380.

Mauricio Zambrano-Bigiarini, Maurice Clerc, and Rodrigo Rojas. Standard particle swarm optimisation 2011 at CEC-2013: A baseline for future PSO improvements. In *IEEE Congress on Evolutionary Computation (CEC)*, 2013. doi: 10.1109/CEC.2013.6557848.